

DevSecOps – Notes (pg.1)

Mindset, Culture, Practice – DevOps vs DevSecOps

What / Why / How – always understand the "why" before the "how".

Traditional DevOps cycle (security missing):

Code → Build → Test → Deploy

↳ Security was applied only at the end – a "bolt-on", too late to be effective.

DevSecOps cycle (security embedded at every stage):

Code(Sec) → Build(Sec) → Test(Sec) → Deploy(Sec)

"Shift Left" ★

- Security is moved to the beginning of the pipeline instead of the end.
- The earlier a vulnerability is caught, the cheaper and faster it is to fix.
- "Left" = early stage (Code); "Right" = late stage (Deploy).

DevOps = Speed + Automation

DevSecOps = Speed + Automation + Security built-in (not bolted on)

Pipeline Stages and Their Security Checks

Stage	Security Check	Example Tool
Code	SAST	Bandit, flake8
Build	Container/Image Scan	Trivy
Test	DAST	OWASP ZAP
Deploy	Dependency/Secret Scan	GitLeaks

SAST (Static Application Security Testing)

- Analyses the source code *without* executing it.
- Detects: SQL Injection risks, code smells, poor coding practices.
- Example tools: Bandit (Python), flake8/ESLint (linters).

Container Scan

- Dockerfile is checked using hadolint.
- The built image is checked using Trivy.

DAST (Dynamic Application Security Testing)

- Tests the application while it is *actually* running, simulating real attacks.
- Identifies runtime vulnerabilities.
- Reference standard: OWASP Top 10 – e.g. the well-known Log4j (Java library) vulnerability.

continued on pg.2 →

Trivy - Image Vulnerability Scan

Trivy compares the image contents against its own **CVE Database**.

Trivy Database Output (example):

CVEs detected -

Python 3.11 (Image) → vulnerable

zip - 1.11 (corrupted / vulnerable)

Python 3.11 → Application ==> [CORRUPT]

Understanding Severity Levels:

Severity	Recommended Action
CRITICAL	Block the build/deploy - must be fixed immediately
HIGH	A patch is usually available - should be fixed promptly
MEDIUM	Acceptable for now - track but do not block
LOW	Can generally be ignored

→ In production pipelines, CI is configured to fail on **CRITICAL/HIGH** findings (a security "gate"), while MEDIUM/LOW are simply reported.

CI / CD Breakdown - DevSecOps Version

CI (Continuous Integration):

- Code Lint & Static/Dynamic Analysis
- Docker Lint (hadolint)
- Secret/Dependency Scan (GitLeaks)
- Build & Push
- Image Scan (Trivy)

CD (Continuous Deployment):

- Deploy the application
- Onto a server (a hosting environment is required)

CI: Lint → SAST/DAST → Docker Lint → Secret Scan → Build & Push → Image Scan



CD: Deploy application → Server

continued on pg.3 →

GitHub Actions – Workflow Files (.github/workflows/)

Mapping each theoretical concept to the actual repository files:

File	Corresponds To	Remarks
code-quality.yml	SAST / Lint	npm install --legacy-peer-deps fix
code-tests.yml	Test Stage	Unit tests executed
dast.yml	DAST	Dynamic scan added
dependency-scan.yml	Dependency Check	Third-party library scan
secret-scanning.yml	Secret Leak Check	GitLeaks
docker-push.yml	Build & Push	Completes CI image push
docker-scans.yml	Image Scan	Trivy scan before deployment
deploy.yml	CD Stage	Actual deployment to server
sonar-scan.yml	SAST (SonarQube)	Code smells, security hotspots
matrix.yml	Build Matrix	Multi-version testing
devsecops.yml	Master Pipeline	Orchestrates all of the above (see next page)

code-quality → code-tests → sonar-scan (SAST)



dependency-scan + secret-scanning



docker-push (build & push image)



docker-scans (Trivy)



dast (runtime scan)



deploy (CD → server)

continued on pg.4 – devsecops.yml syntax →

devsecops.yml – Master Pipeline Syntax ★ explained line by line

1) Header / Trigger Section:

```
name: DevSecOps           // the workflow's display name (visible in the Actions tab)
on:
  push:
    branches: [master]    // triggers automatically on every push to master
```

↳ The comment "Goal: To Run End-to-End DevSecOps CI Pipeline" documents the file's overall purpose.

2) jobs: – each entry is a "job" that calls a separate workflow file

```
jobs:
  code-quality:
    uses: ./github/workflows/code-quality.yml
```

↳ **uses:** means this job does not run logic itself – it simply calls another reusable workflow file, similar to a function call.

3) Independent jobs (can run in parallel – no dependencies):

- code-quality → calls code-quality.yml
- secret-scanning → calls secret-scanning.yml
- dependency-checks → calls dependency-scan.yml
- docker-checks → calls docker-scans.yml (secrets: inherit)
- sonar-qube → calls sonar-scan.yml (secrets: inherit)
- code-tests → calls code-tests.yml

```
docker-checks:
  uses: ./github/workflows/docker-scans.yml
  secrets: inherit
```

↳ **secrets: inherit** – passes all secrets (API keys, tokens) available to the parent workflow down to the called child workflow, without redefining them.

↳ Note: only jobs that require authentication (Docker, SonarQube) use "secrets: inherit"; simple checks (lint, dependency scan, code-quality) do not need it.

Example – devboard project: devsecops.yml (top half)

```
# Goal - To Run End to End DevSecOps CI Pipeline
name: DevSecOps
on:
  push:
    branches: [master]
jobs:
  ## ---- CI ----
  # Code Stage
  code-quality:
    uses: ./github/workflows/code-quality.yml
  # Secret Scanning
  secret-scanning:
    uses: ./github/workflows/secret-scanning.yml
  # Dependency Checks
  dependency-checks:
    uses: ./github/workflows/dependency-scan.yml
  # Docker Checks
  docker-checks:
    uses: ./github/workflows/docker-scans.yml
    secrets: inherit
  # SonarQube
  sonar-qube:
    uses: ./github/workflows/sonar-scan.yml
    secrets: inherit
  # Code Tests
  code-tests:
    uses: ./github/workflows/code-tests.yml
```

continued on pg.5 →

devsecops.yml (continued) – The Role of "needs:"

4) docker-push – waits for all CI checks to succeed

```
docker-push:  
  uses: ./github/workflows/docker-push.yml  
  needs: [code-quality, code-tests, sonar-qube,  
         docker-checks, dependency-checks, secret-scanning]  
  secrets: inherit
```

↳ **needs: []** specifies that the `docker-push` job will run only once all listed jobs have completed successfully.

↳ In other words: if SAST, secret scanning, or SonarQube analysis fails, the image will **not** be built or pushed. This dependency chain is the real security "gate" of DevSecOps.

```
code-quality \  
code-tests  \  
sonar-qube  → (all pass) → docker-push  
docker-checks /  
dependency-checks /  
secret-scanning /
```

5) CD Section – deploy and dast (run after push)

```
deploy:  
  needs: [docker-push]  
  uses: ./github/workflows/deploy.yml  
  secrets: inherit  
  
dast-scan:  
  needs: [deploy]  
  uses: ./github/workflows/dast.yml  
  secrets: inherit
```

↳ **deploy** runs only once `docker-push` has completed (the image is available in the registry).

↳ **dast-scan** runs last, because DAST requires the application to be live and running – so a scan is only possible after deployment.

Overall dependency chain in one line:

```
[code-quality + code-tests + sonar-qube + docker-checks + dependency-checks + secret-scanning] (parallel)  
      ↓ (needs)  
docker-push  
      ↓ (needs)  
deploy  
      ↓ (needs)  
dast-scan
```

Example – devboard project: devsecops.yml (bottom half)

```
# push-to-docker-hub  
docker-push:  
  uses: ./github/workflows/docker-push.yml  
  needs: [code-quality, code-tests, sonar-qube, docker-checks, dependency-checks, secret-scanning]  
  secrets: inherit  
  
## ----- CD -----  
deploy:  
  needs: [docker-push]  
  uses: ./github/workflows/deploy.yml  
  secrets: inherit  
  
dast-scan:  
  needs: [deploy]  
  uses: ./github/workflows/dast.yml  
  secrets: inherit
```

Key Terms – Quick Reference:

- **uses:** → specifies which reusable workflow file to call
- **needs:** → specifies which jobs must succeed first (dependency)
- **secrets: inherit** → passes the parent's secrets to the child workflow
- **on: push: branches:** → defines the trigger condition
- CI jobs run in parallel as a gate, CD jobs run sequentially afterward

Trigger Types – Theory:

Trigger	Meaning
<code>on: push</code>	Runs automatically on a push to the specified branch
<code>on: workflow_call</code>	The file does not trigger itself – it is invoked by another workflow (reusable)
<code>on: workflow_dispatch</code>	Triggered manually from the Actions tab – used by <code>matrix.yml</code>

↳ This is why every child workflow (`code-quality`, `code-tests`, `dast`, etc.) uses **workflow_call** – they are only executed via `devsecops.yml`, not independently.

code-quality.yml - SAST Stage

Goal: Perform Static Application Security Testing on the codebase

Steps (in execution order):

1. **Code Checkout** - fetches the repository onto the runner (always the first step of any job)
2. **Setup Node.js (v20)** - prepares the Node environment for the frontend, with npm caching enabled for speed
3. **npm install --legacy-peer-deps** - installs dependencies while bypassing strict peer-dependency conflicts
4. **npm run lint** - checks JS/TS code style and surfaces basic issues (similar to ESLint)
5. **Setup Go (v1.23)** - prepares the Go environment for the backend
6. **go fmt** - enforces standard Go code formatting
7. **go vet** - performs static analysis to detect suspicious constructs/bugs

Theory:

- **working-directory**: specifies which folder a command runs in (frontend/backend are kept separate in this mono-repo).
- Lint + go vet/fmt together represent a basic form of **SAST** - the code is analysed, not executed.
- **cache**: npm avoids re-downloading dependencies every run, keeping the pipeline fast.

Example - devboard project: code-quality.yml

```
# Goal - I have to do SAST for my Code
name: Code Quality
on:
  workflow_call:
jobs:
  sast:
    runs-on: ubuntu-latest
    steps:
      - name: Code Checkout
        uses: actions/checkout@v7
      - name: Setup Node.js
        uses: actions/setup-node@v6
        with:
          node-version: '20'
          cache: npm
          cache-dependency-path: frontend/package-lock.json
      - name: Install npm Packages
        run: npm install --legacy-peer-deps
        working-directory: frontend
      - name: Run Linter
        run: npm run lint
        working-directory: frontend
      - name: Setup Go
        uses: actions/setup-go@v6
        with:
          go-version: '1.23'
          go-version-file: 'go.mod'
          cache-dependency-path: go.sum
      - name: Run Go Formatter
        run: go fmt
        working-directory: backend
      - name: Run Go Vet
        run: go vet
        working-directory: backend
```

code-tests.yml – Test Stage

Goal: Verify functional correctness through unit tests

Steps:

1. Checkout Code
2. Setup Go + Setup Node.js (both environments, since this is a full-stack application)
3. Install Dependencies (npm install --legacy-peer-deps) – frontend
4. **npm run test** – executes frontend unit tests
5. **go test** – executes backend unit tests

Theory:

- This differs from SAST – here the code is actually executed against defined test cases.
- If any test fails, the job fails, and downstream jobs such as docker-push (which lists this in "needs:") will not proceed.
- Both frontend and backend coverage are handled sequentially within this single job.

Example – devboard project: code-tests.yml

```
name: Code Tests
on:
  workflow_call:
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout Code
        uses: actions/checkout@v7
      - name: Setup Go
        uses: actions/setup-go@v6
        with:
          go-version: '1.23'
          go-version-file: 'go.mod'
          cache-dependency-path: go.sum
      - name: Setup Node.js
        uses: actions/setup-node@v6
        with:
          node-version: '20'
          cache: npm
          cache-dependency-path: frontend/package.json
      - name: Install Dependencies
        run: npm install --legacy-peer-deps
        working-directory: frontend
      - name: Run Tests
        run: npm run test
        working-directory: frontend
      - name: Run Tests
        run: go test
        working-directory: backend
```

dependency-scan.yml – Dependency Checks

Goal: Verify that packages/dependencies have been audited for security issues

Steps:

1. Checkout + Setup Go
2. **go install govulncheck** – installs Go's official vulnerability-scanning tool
3. **govulncheck ./...** – scans all Go packages and writes the results to a report file (`continue-on-error: true`)
4. Setup Node
5. **npm audit** – checks node_modules for known vulnerable packages
6. **Upload Artifact** – saves the report to the Actions tab for later review (5-day retention)

Theory:

- `continue-on-error: true` – even if govulncheck finds vulnerabilities, the job does not fail it simply produces a report.
- `if: always()` – ensures the artifact is uploaded even if a prior step fails, so the report is never lost.
- This step represents **SCA** (Software Composition Analysis) – checking third-party libraries rather than the application's own code.

Example – devboard project: dependency-scan.yml

```
# Goal – I have to ensure the packages & dependencies have been audited for Security
name: Dependency scan
on:
  workflow_call:
jobs:
  dependency-scan:
    runs-on: ubuntu-latest
    steps:
      - name: Code Checkout
        uses: actions/checkout@v7
      - name: Setup Go
        uses: actions/setup-go@v6
        with:
          go-version: 1.22
          go-version-file: 'go.mod'
          cache-dependency-path: go.sum
      - name: Install Govulncheck
        run: go install golang.org/x/vuln/cmd/govulncheck@v1.1.4
      - name: GoVulncheck
        continue-on-error: true
        run: govulncheck ./... > govulncheck-report.txt
        working-directory: backend
      - name: Setup Node
        uses: actions/setup-node@v6
        with:
          node-version: '20'
          cache: npm
          cache-dependency-path: frontend/package-lock.json
      - name: Run Dependency Scan for Node
        run: npm audit
        working-directory: frontend
      - name: Upload Report to Artifacts
        if: always()
        uses: actions/upload-artifact@v4
        with:
          name: dependency-scan-report
          path: reports/dependency-report.txt
          retention-days: 5
```


docker-scans.yml – Docker Checks (Build-Stage Security)

Goal: Ensure both the Dockerfile and the built image are security-tested

Understanding strategy: matrix

- **matrix: folders: [backend,frontend]** – runs the same job twice, once for backend and once for frontend, in parallel.
- **fail-fast: false** – if the backend run fails, the frontend run is not cancelled; both proceed independently.

Steps (repeated per folder):

1. Checkout
 2. **hadolint** – lints the Dockerfile against best practices (e.g. avoid running as root, optimise layers)
 3. Docker login (DockerHub)
 4. **Docker Build** – builds the image locally (not yet pushed)
 5. **Trivy Scan** – checks the built image for CVEs
 - ↳ **exit-code: 1** = fails the job if vulnerabilities are found
 - ↳ **severity: CRITICAL** = only critical-level findings are counted
 - ↳ **ignore-unfixed: true** = ignores vulnerabilities with no available patch (avoids wasted effort)
- ↳ This job uses **secrets: inherit** in devsecoops.yml because it requires DockerHub authentication.

Example – devboard project: docker-scans.yml

```
# Goal - Ensure the docker files and Docker images are security Tested
name: Docker Scan
on:
  workflow_call:
jobs:
  docker-file-lint-and-scan:
    runs-on: ubuntu-latest
    strategy:
      fail-fast: false
      matrix:
        folders: [backend,frontend]
    steps:
      - name: Code Checkout
        uses: actions/checkout@v7
      - name: ${{ matrix.folders }} Dockerfile Lint
        uses: hadolint/hadolint-action@v3.10
        with:
          dockerfile: ${{ matrix.folders }}/Dockerfile
      - name: Docker Setup [Login]
        uses: docker/login-action@v1
        with:
          username: ${{ vars.DOCKERHUB_USERNAME }}
          password: ${{ secrets.DOCKERHUB_TOKEN }}
      - name: Docker Build ${{ matrix.folders }}
        run: docker build -t ${{ vars.DOCKERHUB_USERNAME }}devboard-{{ matrix.folders }}latest .
        working-directory: ${{ matrix.folders }}
      - name: Run Trivy vulnerability scanner
        uses: aquasecurity/trivy-action@v0.36.0
        continue-on-error: true
        with:
          image-ref: ${{ vars.DOCKERHUB_USERNAME }}devboard-{{ matrix.folders }}latest
          format: 'table'
          exit-code: '1'
          ignore-unfixed: true
          vuln-type: 'os,library'
          severity: 'CRITICAL'
```

docker-push.yml – Build & Push (Final CI Step)

Goal: Push the final verified image to DockerHub

Steps:

1. Checkout
2. Docker login
3. **docker/build-push-action** – builds and pushes the image in a single step
 - ↳ **context**: selects the relevant Dockerfile folder (backend/frontend, via matrix)
 - ↳ **push: true** – this flag actually triggers the push; otherwise only a build occurs
 - ↳ **tags**: defines the image name and version (e.g. devboard-frontend:latest)

Theory:

- This job only runs once **all** jobs listed under "needs:" have passed (code-quality, tests, SonarQube, docker-checks, dependency-checks, secret-scanning).
- In other words: if any security check fails, the push is **blocked** – this is the actual enforcement point ("gate") of DevSecOps.
- The same matrix approach is used here, producing separate backend and frontend images.

Example – devboard project: docker-push.yml

```
# Goal Push the images to DockerHub
name: Docker Push
on:
  workflow_call:
jobs:
  frontend:
    runs-on: ubuntu-latest
    strategy:
      fail-fast: false
    matrix:
      folders: [backend, frontend]
    steps:
      - name: Checkout Code
        uses: actions/checkout@v7
      - name: Docker Setup [Login]
        uses: docker/login-action@v4
        with:
          username: ${{ vars.DOCKERHUB_USERNAME }}
          password: ${{ secrets.DOCKERHUB_TOKEN }}
      - name: Docker Build and Push
        uses: docker/build-push-action@v7
        with:
          context: ../../${{ matrix.folders }}
          push: true
          tags: ${{ vars.DOCKERHUB_USERNAME }}/devboard-${{ matrix.folders }}:latest
```

secret-scanning.yml

Goal: Ensure no secrets (API keys, passwords, tokens) are leaked in the code

Steps:

- Checkout (**fetch-depth: 0**) - downloads the complete Git history, since a secret may have been committed previously and later removed.
- **GitLeaks Scanner** - uses pattern matching to detect known secret formats (AWS keys, private keys, tokens).
- ↳ Theory: this is a "regex + entropy based" scanner; scanning history is essential, otherwise older leaks could be missed.

sonar-scan.yml

Goal: Assess code coverage and security hotspots (advanced SAST)

Steps:

- Checkout
- **SonarQube Scan Action** - sends code for analysis to an external SonarQube server, authenticated via SONAR_TOKEN and SONAR_HOST_URL secrets.
- ↳ Theory: SonarQube goes beyond basic linting - it reports code smells, security hotspots, duplication percentage, and a maintainability rating. This makes it a more advanced SAST layer than tools like Bandit or flake8.

continued on pg.12 →

dast.yml – OWASP ZAP DAST Scan

Goal: Confirm the running application is free of OWASP Top 10 issues

Steps:

- Checkout
- **Wait for service to be healthy** – a curl-based retry loop that waits until the application responds on port 8080 (60-second timeout), since the app needs time to start after deployment.
- **OWASP ZAP Baseline Scan** – sends attack-like requests to the live URL and checks against the OWASP Top 10 categories.
- ↳ **fail_action: false** – the pipeline does not fail even if issues are found; only a report is generated (this is a team decision; can be set to "true" for stricter enforcement).
- ↳ Theory: this step is only possible after deployment, which is why "needs: [deploy]" appears in the master pipeline.

deploy.yml – CD Stage

Goal: Deploy the verified images to the live server once CI has passed

Steps:

- **runs-on: self-hosted** – this job runs on a privately managed server/EC2 instance rather than a GitHub-hosted runner.
- `cp .env.example .env` – prepares the environment file.
- Docker login
- **docker compose pull + up -d** – pulls the latest pushed images and starts the containers in detached (background) mode.
- ↳ Theory: because the runner is self-hosted, this job executes directly on the target server, allowing containers to be updated immediately via Docker Compose.

continued on pg.13 →

matrix.yml – Go Linter (Multi-Version)

Goal: Lint the Go codebase across multiple Go versions

Steps:

- on: `workflow_dispatch` – triggered manually (not part of the automated master pipeline).
- matrix: go: ['1.22', '1.23', '1.24'] – the same job runs three times, once per Go version.
- go fmt + go vet – executed for each version.

↳ Theory: this verifies that the code remains **compatible** across multiple Go versions – important in larger teams where developers may use different toolchain versions.

secrets vs vars – clarifying the difference:

Type	Example	Usage
secrets.*	DOCKERHUB_TOKEN, SONAR_TOKEN, GITHUB_TOKEN	Sensitive – encrypted, never exposed in logs
vars.*	DOCKERHUB_USERNAME	Non-sensitive configuration – can be stored as plain text

Overall Pipeline Summary:

CI (parallel gate): code-quality + code-tests + sonar-qube + docker-checks + dependency-checks + secret-scanning

↓ (all pass)

docker-push (image → DockerHub)

↓

deploy (self-hosted server, docker compose up)

↓

dast-scan (live application tested via OWASP ZAP)

– End of Notes ♦